

Payment API Mocking

Senior SDET Interview Question & Explanation

Interview question

How can you mock a payment API when the endpoints are the same?

What the interviewer is testing: Whether you understand how to simulate different downstream payment-service responses when the HTTP method and URL do not change.

1. Understand the problem

Assume the application calls the same endpoint for every payment scenario:

```
POST https://payment.company.com/api/payments
```

You need to test successful payment, declined payment, insufficient funds, timeout and service-unavailable scenarios. Since the endpoint is identical, the mock server needs another way to decide which response to return.

2. Main solution: request-based matching

Configure multiple mock rules for the same HTTP method and URL. Each rule matches a distinguishing part of the incoming request, such as a JSON field, a request header, or a scenario identifier. The matching rule determines the response.

Example request A — successful payment

```
{ "cardNumber": "4111111111111111", "amount": 1000, "currency": "INR" }
```

Example response A

```
{ "status": "SUCCESS", "transactionId": "TXN123", "message": "Payment successful" }
```

Example request B — declined payment

```
{ "cardNumber": "4000000000000002", "amount": 2000, "currency": "INR" }
```

Example response B

```
{ "status": "DECLINED", "transactionId": null, "message": "Payment declined" }
```

Both requests use POST /api/payments. The mock server distinguishes them by matching request data, not by using different endpoints.

3. Other ways to select a mock response

Matching method	How it works	When useful
Request payload	Match a field such as amount, currency, payment type or a test reference.	When test inputs naturally distinguish scenarios.
Custom header	Send a test-only header such as X-Mock-Scenario: PAYMENT_DECLINED.	When tests need explicit control over the response.
Mock scenario/configuration	Set up the mock server with a named scenario before a test runs.	When the framework manages test setup and teardown.

Important: custom mock headers or scenario switches should be restricted to test environments. Do not expose a production mechanism that lets a caller force a payment outcome.

4. WireMock example — same endpoint, different payload matchers

WireMock supports matching requests by method, URL, headers and body patterns. The following are illustrative stub mappings. Register each mapping with WireMock's mappings endpoint or place them in its mappings directory.

Stub A — match amount 1000 and return success

```
{
  "request": {
    "method": "POST",
    "url": "/api/payments",
    "bodyPatterns": [{
      "matchesJsonPath": "$[?(@.amount == 1000)]"
    }]
  },
  "response": {
    "status": 200,
    "jsonBody": {
      "status": "SUCCESS",
      "transactionId": "TXN123"
    },
    "headers": { "Content-Type": "application/json" }
  }
}
```

Stub B — match amount 2000 and return declined

```
{
  "request": {
    "method": "POST",
    "url": "/api/payments",
    "bodyPatterns": [{
      "matchesJsonPath": "$[?(@.amount == 2000)]"
    }]
  },
  "response": {
    "status": 400,
    "jsonBody": {
      "status": "DECLINED",
      "message": "Payment declined"
    },
    "headers": { "Content-Type": "application/json" }
  }
}
```

The same POST URL is used in both stubs. WireMock selects the stub whose request matcher matches the incoming request. In a real project, make matchers specific enough to avoid overlapping or ambiguous stubs.

5. Where mocking fits in the test strategy

Test level	Suggested approach	What it validates
Unit test	Mockito or an equivalent test double	Application logic in isolation.
Service integration test	WireMock / service virtualisation	How the application handles controlled downstream responses.
End-to-end test	Payment provider sandbox	Integration with the provider's test environment.

Key distinction: A mock verifies how your application behaves when it receives a simulated payment response. It does not prove that the real payment provider is functioning correctly.

6. Interview-ready answer

When multiple payment scenarios use the same API endpoint, I would use request-based matching in a mock server such as WireMock. For example, for POST /api/payments, I can configure multiple stubs for the same method and URL, but differentiate them using request payload attributes, headers or scenario identifiers. Based on the matching criteria, WireMock returns the corresponding predefined response. At framework level, I would keep the mock configuration reusable and data-driven, and ensure each test has isolated setup and cleanup. For service integration testing, I would use service virtualisation; for end-to-end testing, I would use the payment provider's sandbox. This lets us test positive and negative payment flows without depending on the live third-party service.

7. Your answer — evaluation

Your answer: **payload**

Evaluation: Correct. Matching the request payload is a valid and common way for a mock server to distinguish scenarios when the endpoint is the same. To make your answer stronger in an interview, explain that the mock server matches a specific field or combination of fields in the JSON body and returns the corresponding stubbed response. Also mention headers or scenario configuration as alternatives.

A stronger short answer

I would configure multiple stubs for the same HTTP method and URL, and use request matching on a payload field such as a payment type or test reference to select the expected response. For example, one payload can match a success stub and another can match a declined-payment stub. I would use WireMock and keep the stubs isolated and data-driven.

8. Follow-up questions to practise

- What happens if two WireMock stubs match the same request?
- How would you simulate a payment timeout or connection reset?
- How would you prevent one test's mock configuration from affecting another test?
- What is the difference between mocking with Mockito and WireMock?
- Why should end-to-end payment tests also use a provider sandbox?

Reference: WireMock documentation — Request Matching: <https://wiremock.org/docs/request-matching/>